Paxos and Co. for fault-tolerance

Keeping physical copies (replicas) of a logical data item ”consistent” despite failures and keep the data item available

Consensus Problem: The classical agreement problem

- Model:
  - ☆ set of processes P_1, …. P_N
  - ☆ communication reliable but asynchronous
    - each message sent is eventually delivered to all correct recipients
    - (retransmission, network partitions eventually heal, ….)
  - ☆ processes only fail by crashing and then stop executing
    - correct process: exhibits no failures at any point in the execution under consideration
    - faulty process:
      - ▲Non-failed before failure
      - ▲Crashed afterwards

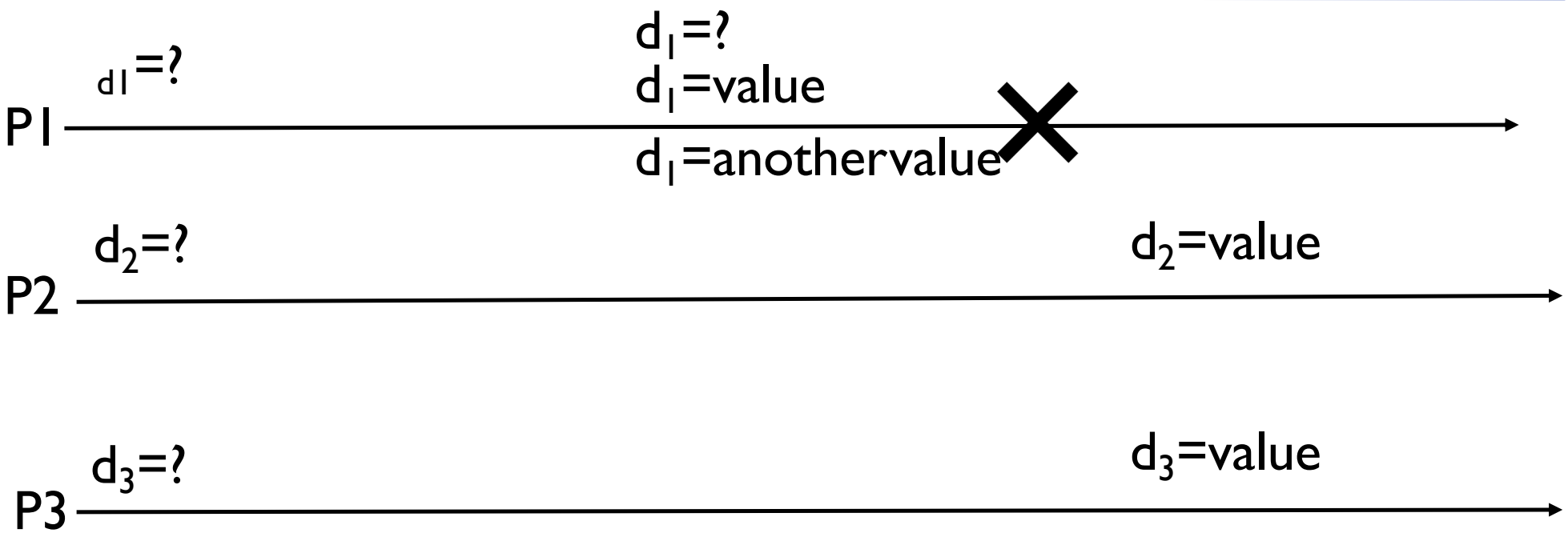
Failure detector

- Module on each node
- takes as input identifier of p
- returns either “suspected” or “unsuspected”
- implementation: if failure detector has received message from p within last x time-units, then unsuspected, otherwise suspected
  - ☆ Heartbeat messages if necessary
- imperfect:
  - ☆ Processes might be falsely suspected to have crashed

Consensus Problem Definition

- Goal: processes agree on the value of a decision variable d_i
- Start: every process p_i begins *undecided* and proposes a value v_i.
- Protocol: processes communicate with each other exchanging values.
- End: Each process sets decision variable d_i, enters *decided* state, and does not change d_i anymore.
- Conditions:
  - ☆ *Termination (liveness)*: Eventually each correct process enters decided state
  - ☆ *Agreement (safety)*: The decision variable of all correct processes is the same: if p_i and p_j are correct and have entered the decided state, then $d_i=d_j$
  - ☆ *Integrity*: (different options, here one example)
    - The chosen value must be one of the proposed values
- 100% correct solution in all situations: non-existent [Fischer et al., 1985]

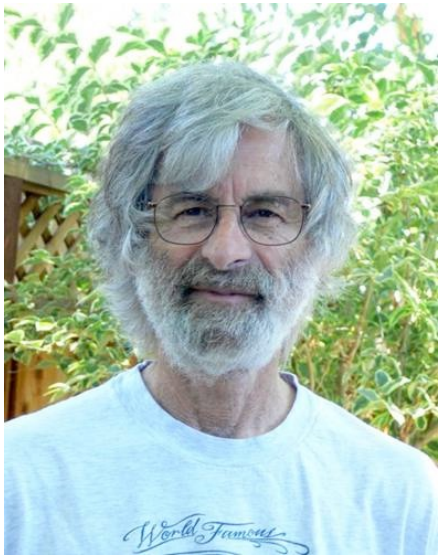
Consensus



The original Paxos: the part-time parlament

ACM TOCS

- ☆ Transactions on Computer Systems
- ☆ Submitted 1990. Accepted 1998



Leslie Lamport

Turing Award 2013

Abstract

Recent archaeological discoveries on the island of Paxos reveal that the parliament functioned despite the peripatetic propensity of its part-time legislators. The legislators maintained consistent copies of the Parliamentary record, despite their frequent forays from the chamber and the forgetfulness of their messengers. The Paxon parliament's protocol provides a new way of implementing the state-machine approach to the design of distributed systems …

Easier Versions: Paxos made simple / Paxos made live

At the same time (and nearly equivalent)

Viewstamped Replication (VR)

- ☆ Barbara Liskov / Brian Oki 1988

Raft

- ☆ Diego Ongar, John Ousterhout et. al. 2014



Barbara Liskov

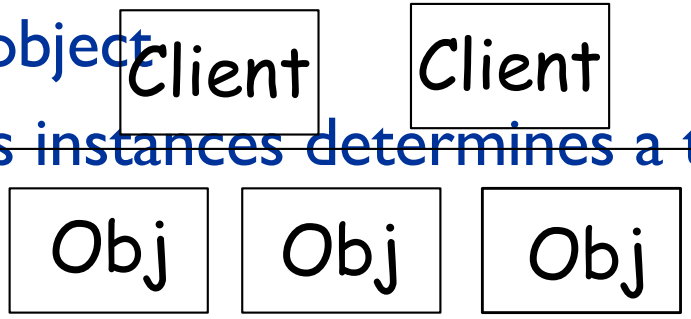
Turing Award

2008

VR provides the same properties as Paxos

Usage: replicated storage

- Replicate a file/database/object etc. on 3 or 5 nodes
- Fault-tolerance: object can be accessed even if replica fails
- Only consider update requests: consistency
  - ☆ all replicas that are up and running must be updated
- Two models:
  - ☆ "Update anywhere": Client can send update request to any replica
  - ☆ "Primary Copy": Client must send update to one specific primary object replica. If it detects failure, it connects to other replica
- Paxos: consensus algorithm that coordinates
  - ☆ An instance of the Paxos algorithm decides on what is the next update on the data object
  - ☆ A sequence of Paxos instances determines a total order on the updates



Consensus vs. Paxos

- Basic Model the same:
  - ☆ set of processes P_1, …. P_N
  - ☆ communication reliable but asynchronous
- Process Failure Model
  - ☆ Consensus
    - correct process (up all the time) vs. faulty process (first up, then crash, then down for ever)
  - ☆ Paxos
    - Crash-Recovery
      - ▲Process fail by crashing
      - ▲Crashed processes will eventually recover
      - ▲Processes can log their state in stable storage
    - But mostly ignore this in this lecture

Consensus vs. Paxos II

- Agreement
  - ☆Consensus: the decision variables of all *correct processes* that have decided, have the same value
  - ☆Paxos: the decision variable of *all processes* that have decided, have the same value
- Handling Impossibility Result
  - ☆safety over liveness (guarantees agreement but not termination)
  - ☆Paxos tries to be ”as live” as possible
  - ☆Paxos uses a coordinator approach (called leader, proposer)
  - ☆if leader crashes, another leader takes over and tries to finish the job → if again crash, another tries to take over, with possibly infinite number of take-overs…

Consensus vs. Paxos

- Paxos decides on one of the proposed values
- Usage: replication of an object x:
  - ▲decide on the next update for all replicas
  - ▲Doesn't matter which update of several concurrent ones is taken as long as all decide on the same

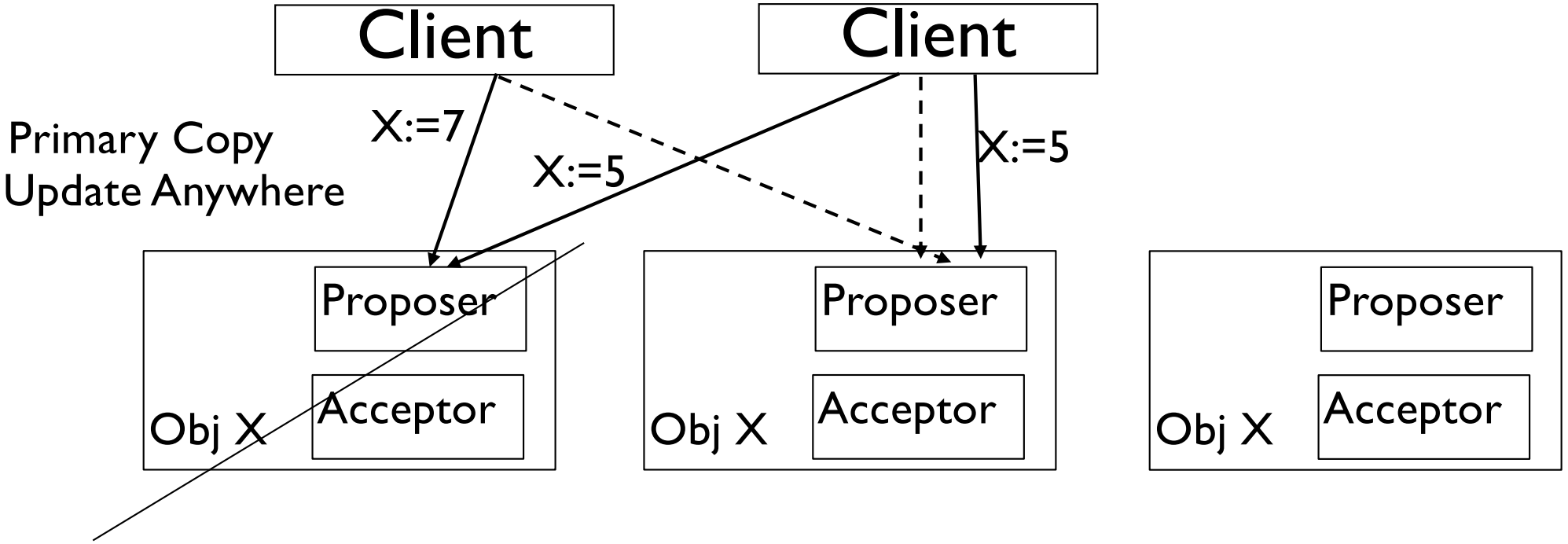
Majority Concept

- Once a *majority* of Paxos servers agrees on one of the proposed values, we have consensus on that value.
  - Because if a majority agrees, there cannot be another majority that would agree on a different value…

Entities

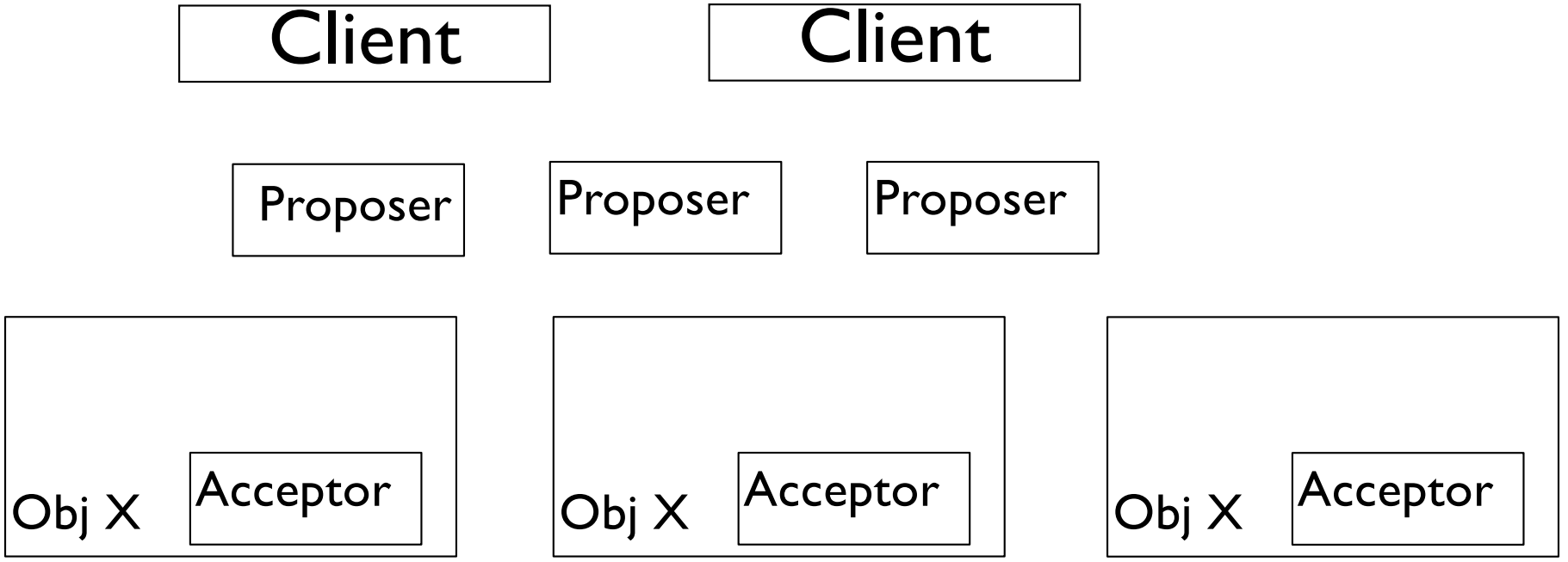
- **Proposers (Leader)**:
  - ☆ Receive requests (values) from clients and try to convince acceptors to accept their proposed values.
  - ☆ Possibly more than one proposer because of fault-tolerance (primary copy) and flexibility (update anywhere)
- **Acceptors**:
  - ☆ Acceptors are the ones who accept proposals and eventually install updates
  - ☆ Several because of fault-tolerance.
  - ☆ For progress in case of failures or concurrency: an acceptor must be able to change their mind and accept a new proposal
  - ☆ For consistency: as soon as a majority has accepted a value, then this value must eventually be installed

Architecture I



- A replicated database has a proposer and an acceptor component
- Each executes a different protocol that together are the Paxos algorithm

Architecture II



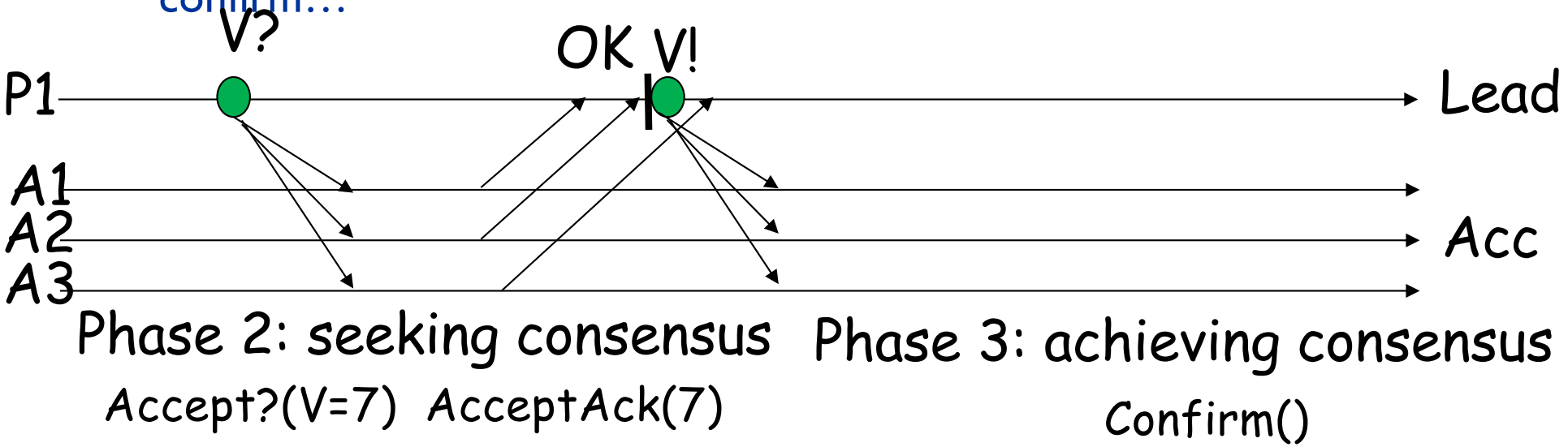
- Proposer protocol independent of acceptor protocol
- Thus, no need to be co-hosted
- A set of proposers could be on different machines
- Majority concept is only necessary for acceptors

Entities

- **Learners**: we ignore them in this lecture
- A server/replica can be proposer and acceptor
  - ☆E.g., in the following proposer p1 and acceptor a1 in the same process with two roles

Paxos Instance with one proposer that never fails

- N acceptors
  - Majority required for consensus
- Leader/proposer/coordinator
  - Presents a consensus value to the acceptors and asks acceptors to accept that value
  - If a majority accepts: confirms success to acceptors
- Careful with out-of-order messages
  - Send confirm after majority but can receive AcceptAck after having send the confirm…

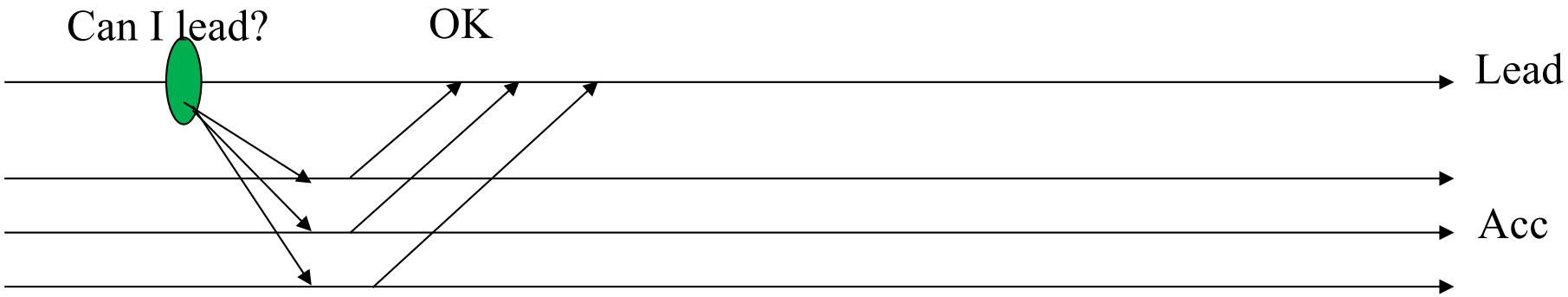


Multiple proposer

- Scenarios
  - ☆ New proposer takes over when old one fails
  - ☆ Clients can submit requests to different proposers
- Paxos is safe with multiple proposers/leaders
  - ☆ Leader election is built in
    - **Phase I: Propose to be leader**
    - Phase II: Ask to accept a value
    - Phase III: confirm decision
  - ☆ If consensus appears stalled, anyone can try to take over as leader
    - Initiate a new phase I
  - ☆ If several proposers are allowed to receive requests from clients, then all try in a phase 1 to be the leader for this Paxos instance
  - ☆ Live-lock can occur and things never terminate….

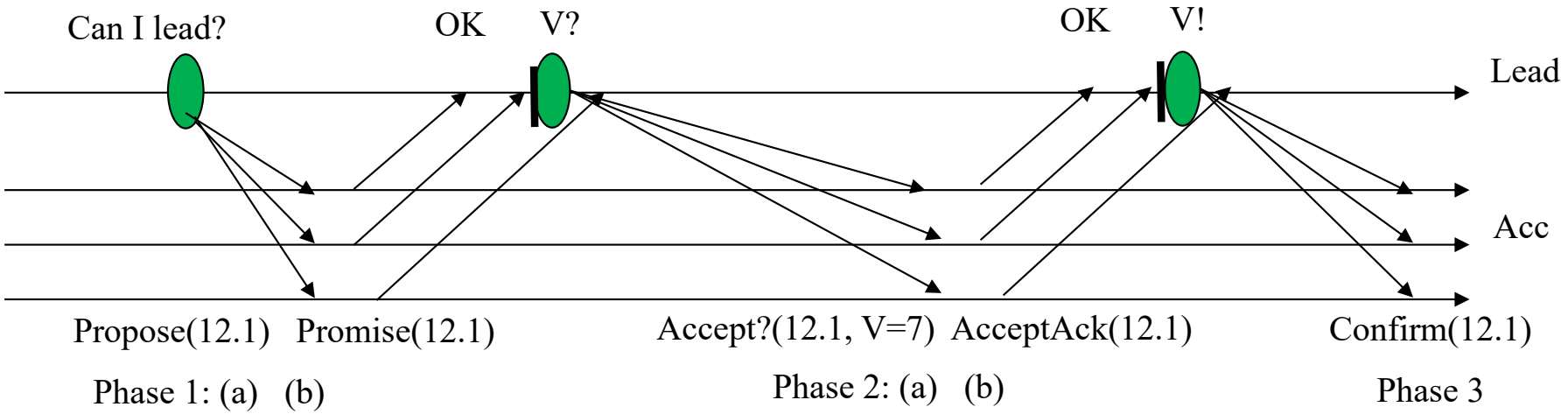
3-phase Paxos with no failure, single proposer

- Phase 1:
  - ☆ Propose request:
    - send propose(ballotID) with unique ballotID to acceptors with request to be the leader
  - ☆ Promise response:
    - If ballotID > maxB // keep track of highest promised ballotID
      - ▲maxB=ballotID
      - ▲return promise(ballotID) (more detailed later)
    - Else refuse (maxB)



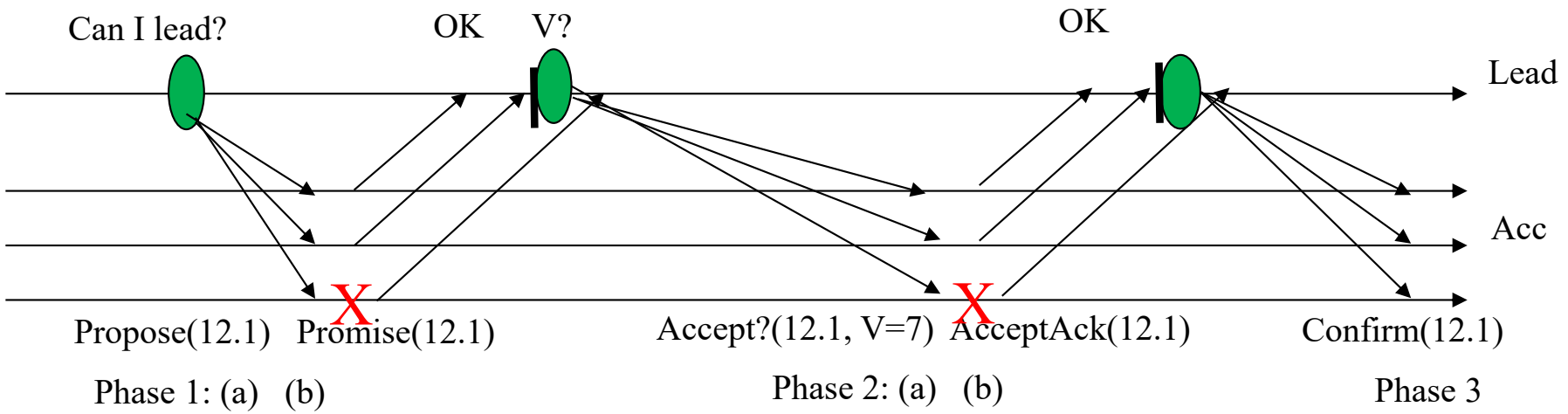
3-phase Paxos with no failure, single proposer

- Phase 2/3
  - ☆ If majority promises (willing to have proposer as leader for this round), then phase II and III
  - ☆ Accept? request, accept response AcceptAck, confirm
    - Piggyback ballotID



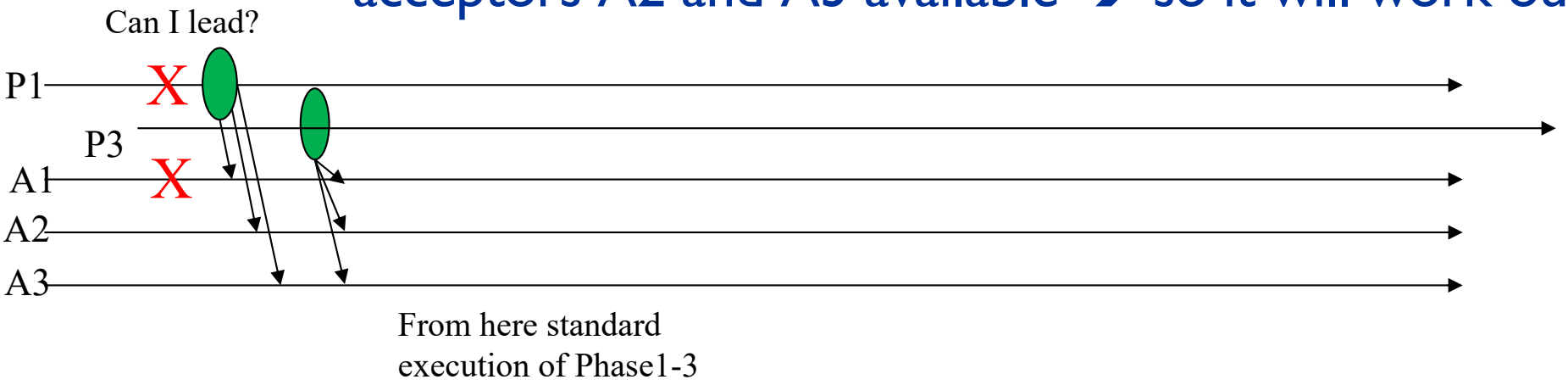
Acceptor Failure

- Acceptor fails
  - ☆ Before sending promise
    - Proposer still receives majority of promises
  - ☆ After promise but before sending AcceptAck
    - Proposer still receives a majority of AcceptAck
- Protocol can handle a minority of acceptors to fail



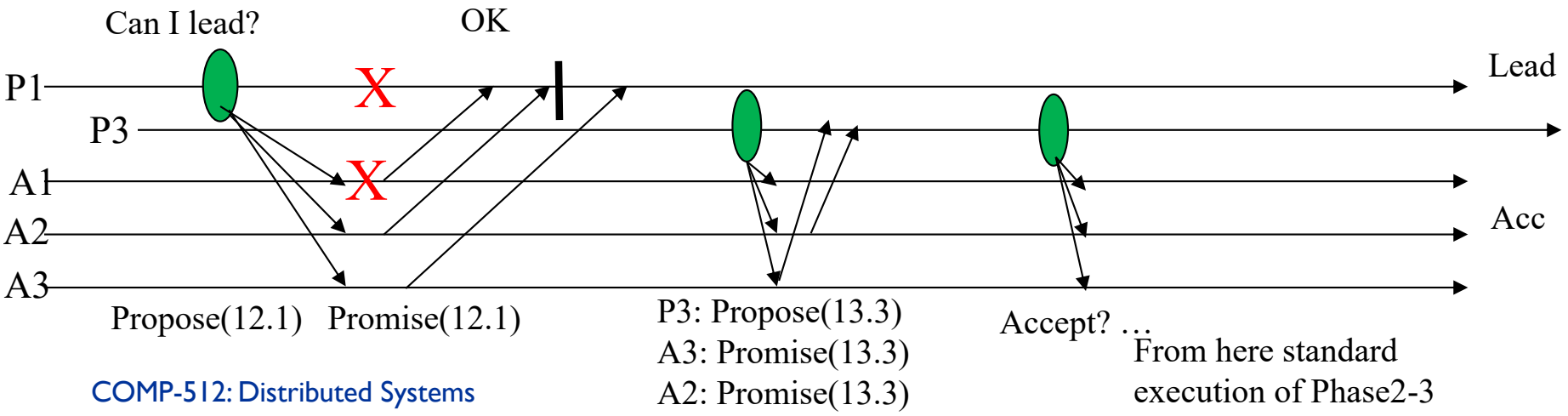
Failure Cases

- Proposer fails
  - ☆ Before sending propose
    - Round doesn't even start; eventually other proposer will take over
    - E.g., Process of A3 becomes Proposer P3 and starts new round; sends e.g., Propose(13.3)
      - ▲if P1 fails together with A1 (same node): Still acceptors A2 and A3 available ➔ so it will work out



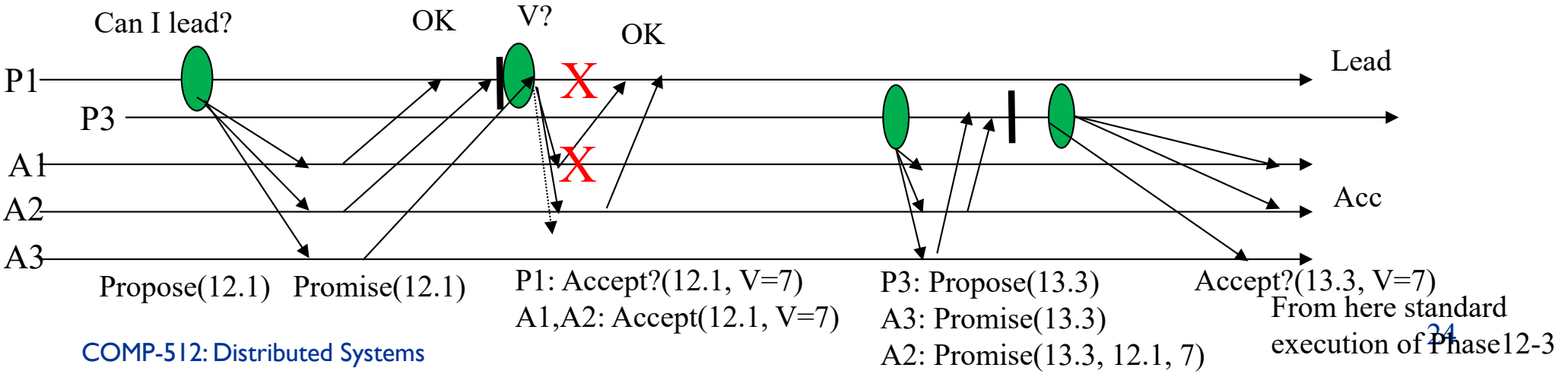
Failure Cases

- Proposer fails
  - ☆ After having sent propose but before sending out Accept?
    - Take-over proposer, starts new round, hopefully chooses higher ballotID for promise
    - Can continue if majority still up



Failure Cases

- Proposer fails
  - ☆ After possibly having sent some Accept? but not yet to all
    - Take-over proposer starts new round chooses higher ballotID for promise
    - **If acceptor has accepted previous values, it returns the one associated with the highest ballotID it has accepted together with that ballotID with promise**
    - Take-over proposer will take this value
    - If there is no such value (none of the alive acceptors has received it), then choose any value



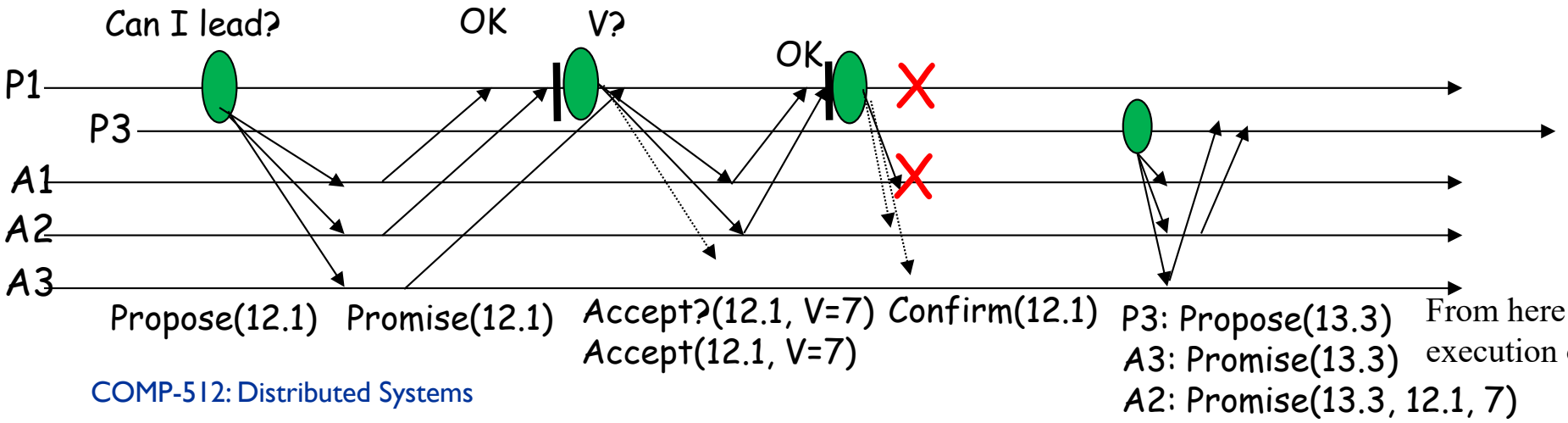
Key Point

- That is:
  - ☆ If the proposer fails after a partial decision has made it to the acceptors (accept phase II), then any acceptor who has received the value will propagate it to the next round, and the next proposer will finish the job with this value
- Why is this important?

Failure Cases

- ☆ Old proposer might have received majority of Accept
- ☆ Old proposer might then have sent already confirm to some acceptors before fail and they have committed
- ☆ Example: P1 has received accept from A1 and A2, sends confirms to A1 and then crashes (and A1, too, as same process!).
- ☆ Thus, A1 has installed value → the others should do

- ☆ Everybody should install the same value



Key Point

- Why does it work
  - ☆ Confirm can only happen when majority has accepted
  - ☆ New proposer must contact a majority – intersection guaranteed (here it's A2). And A2 can inform about accept and value propagates to next round

Algorithm sketch: Proposer

Assumes request V from client

- ☆ choose ballotID
- ☆ send propose(ballotID) to all acceptors
- ☆ wait for promise(ballotID) from acceptors
- ☆ If doesn't receive a majority of promises,
  - ● give up, possibly restart with a new ballotID
- ☆ Else (receive promise(ballotID) or promise(ballotID, bid2, v') from majority of acceptors)
  - ● If there is at least one promise(ballotID, bid2, v')
    - ▲ Let promise(ballotID, bid2, v') be the promise message with the highest value for bid2
    - ▲Send accept?(ballotID,v') to all acceptors
  - ● Else (only promise(ballotID))
    - ▲send accept?(ballotID,V) to all acceptors

Proposer Cont'd

- wait for AcceptAck(ballotID) from acceptors
- If doesn't receive a majority of AcceptAck,
  - ▲give up, possibly restart with a new ballotID
- Else (majority accepts value)
  - ▲This value is now committed
  - ▲Send confirm(ballotID) to all acceptors

Algorithm sketch: Acceptor

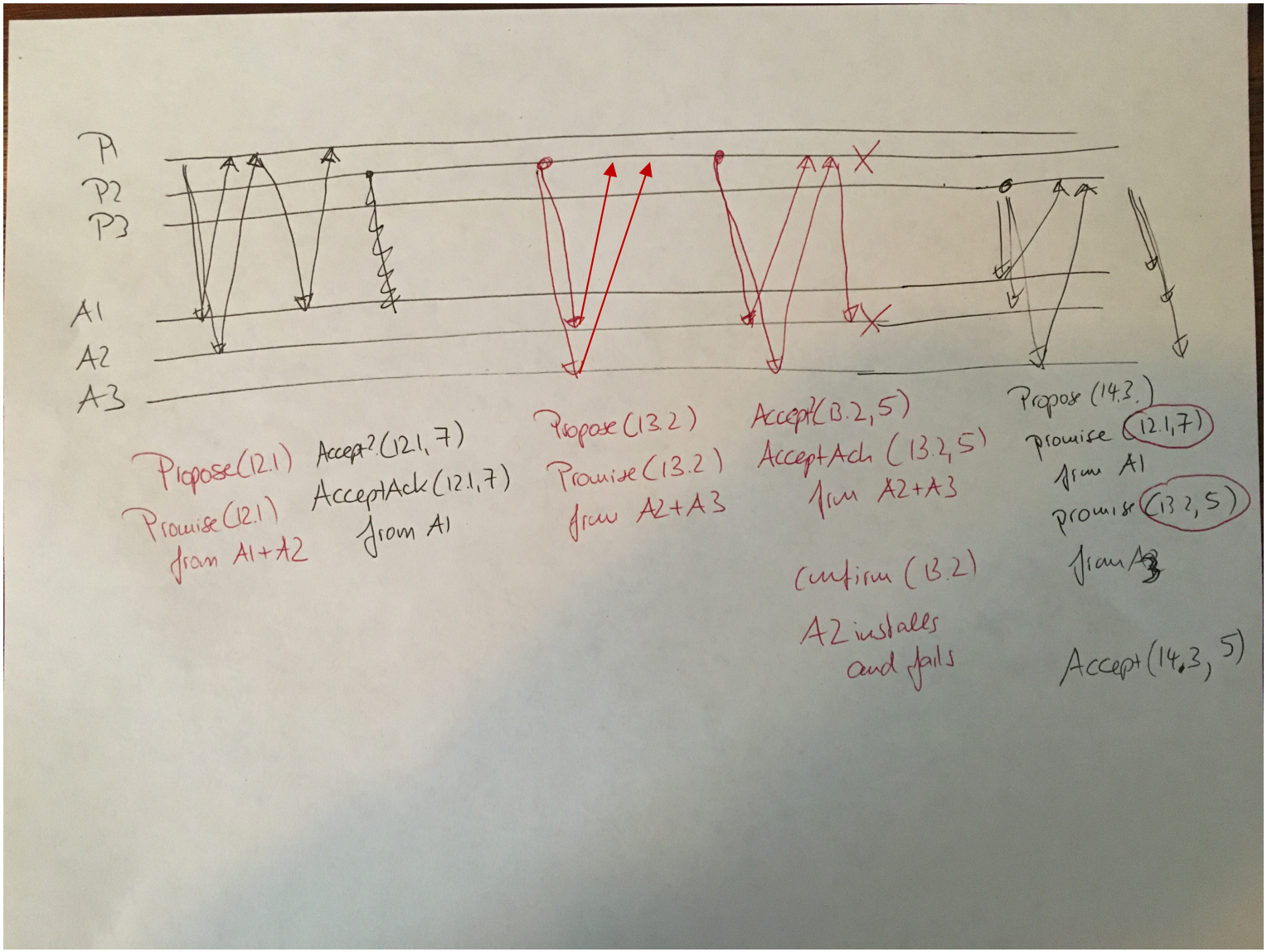
- maxBID = -1, val = NIL, acceptBID = -1
- Upon receiving propose(ballotID)
  - ☆ If ballotID < maxBID
    - Send Refuse(ballotID,maxBID) to proposer
  - ☆ Else (ballotID higher than any seen so far), *promise*
    - If val == NIL
      - ▲send promise(ballotID) to proposer
    - Else
      - ▲ send (promise(ballotID, acceptBID, val) to proposer // send accepted value and the ballotID that asked to accept that value
    - maxBID = ballotID

Acceptor Cont'd

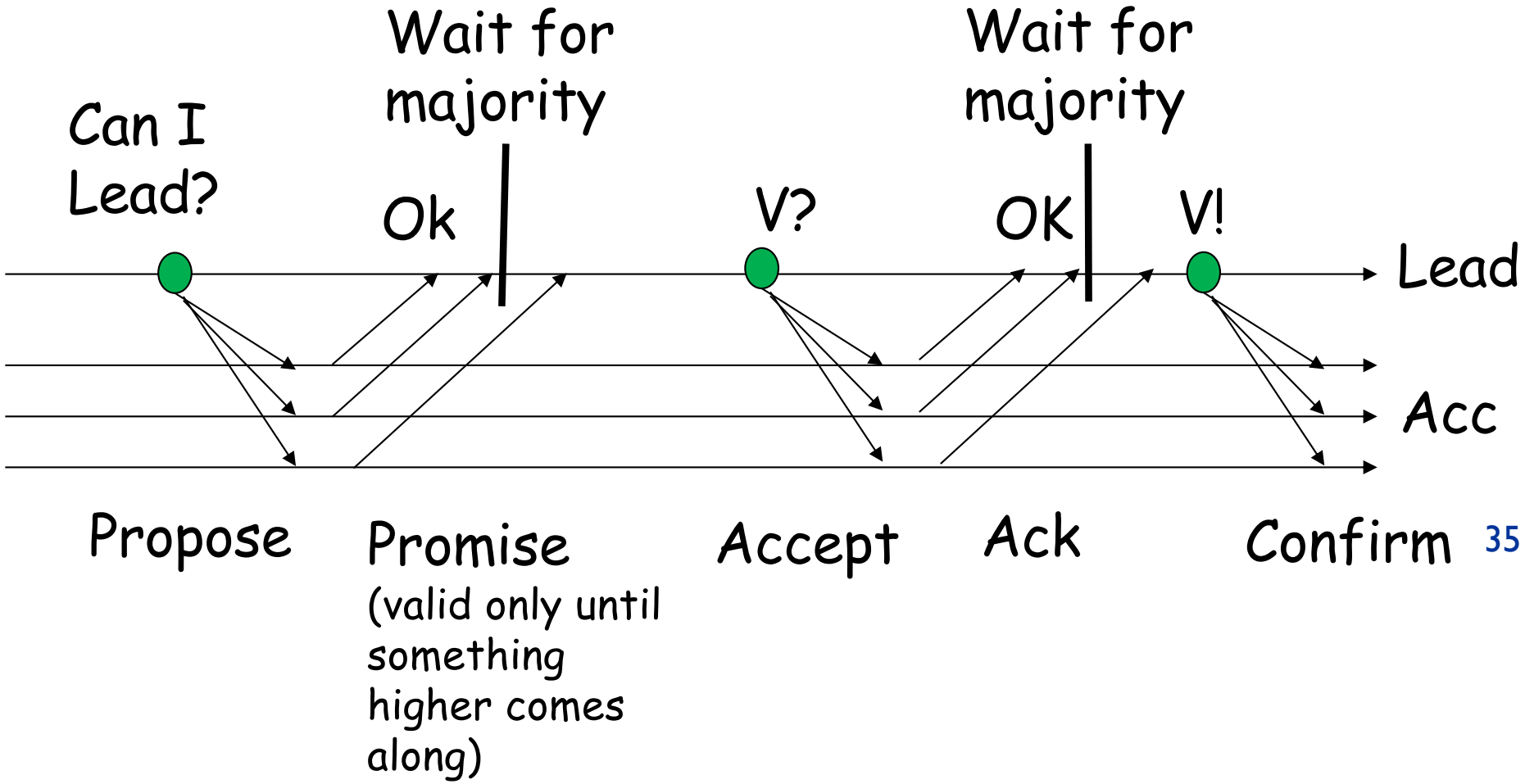
- Upon receiving accept?(ballotID,value)
  - ☆ If ballotID == maxBID
    - val = value (Accept value), acceptBID = ballotID
    - overrides any previously accepted value
    - Send AcceptAck(ballotID) to proposer
  - ☆ Else (has accepted higher ballotID since then)
    - send deny(ballotID,maxBID) to proposer

- Upon receiving confirm(ballotID)
  - ☆ If ballotID == maxBID
    - Install current val as committed value
  - ☆ (note that it might still receive further proposals, etc.!!! And should respond to help others to complete)

- Round: Every time a proposer starts a proposal and initiates message exchange and execution of the protocol
- Instance: the set of rounds that finally lead to have a value accepted and installed at a majority of acceptors.
- One instance with one round: slides 20/21
- One instance with two rounds: slides 22-26


P1
P2
P3
A1
A2
A3
Propose(12.1)
Promise(12.1)
from A1+A2
Accept?(12.1,7)
AcceptAck(12.1,7)
from A1
Propose(13.2)
Promise(13.2)
from A2+A3
Accept(13.2,5)
AcceptAck(13.2,5)
from A2+A3
confirm(13.2)
A2 installs
and fails
Propose(14.3)
promise (12.1,7)
from A1
promise (13.2,5)
from A3
Accept(14.3,5)

A successful Paxos Round/Instance



35

- Where is the consensus point of no return?

Liveness and safety

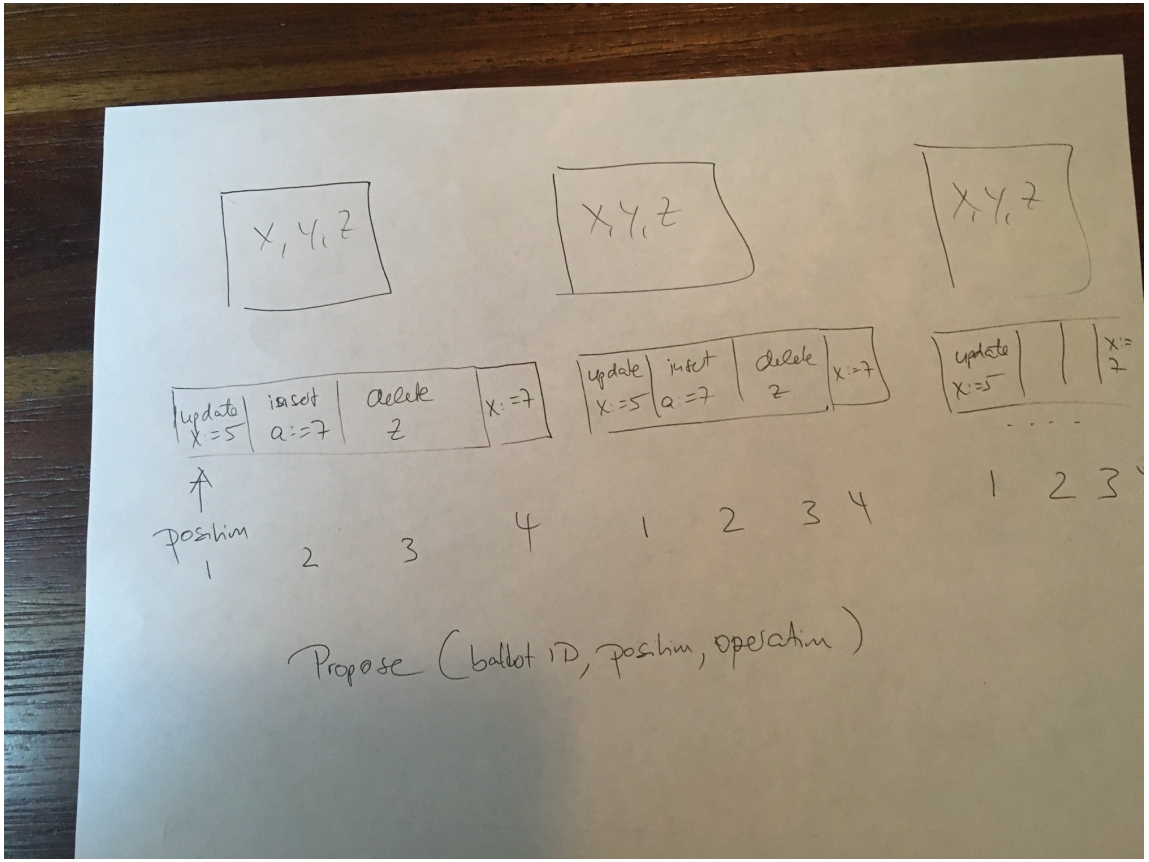
- Liveness requires progress
  - ☆ Should a round not terminate a timed-out acceptor can take over role as proposer and start a new round with higher ballotID
  - ☆ that acceptors are free to accept different values in subsequent rounds
- Safety requires that once a majority have accepted (and with this possibly someone installed the value), no subsequent round changes the value
  - ☆ Accepted values are piggybacked and carried forward to the next round

Need for persistence

- Persistence needed for recovery
  - ☆ Needed for liveness; if you recover, you can continue the protocol
- For example: Acceptor fails after receiving accept and after sending accept ack (phase 11)
  - ☆ It must remember that it has accepted the value
  - ☆ write it to stable storage
  - ☆ So when it recovers and continues in the algorithm, it knows that it has already accepted a value
- Overall:
  - ☆ Logging before every message: 5 logs!
- We discuss cost of logging when we discuss distributed transactions

Multi-Paxos

- Chubby Replicated Database
  - Data items stored in key/value data store
  - Each replica has Replicated log of database commands (insert/update/…)
- Paxos decides on the order requests are appended to log.
- For each position in the log: one Paxos agreement instance
  - (possibly multiple rounds until one round succeeds)
- How do you know in which instance you are (which position in the log)
  - E.g., p1 wants x:=7, p2 wants x:=5, start concurrently
  - P1's value makes it
  - P2 should start a new instance of the protocol to make x:=5 the next update (by proposing to be the leader)
  - How does a process p3 know that this is a new instance???
  - You could have some instance counter / indicate the position in the log that you want to fill – could become another parameter in the message exchange


X, Y, Z
X, Y, Z
X, Y, Z
update X:=5
insert a:=7
delete z
x:=7
update X:=5
insert a:=7
delete z
x:=7
update X:=5
x:=
z
position
1 2 3 4 1 2 3 4 1 2 3
Propose (ballot ID, position, operation)

Stable leader

- Primary/Secondary architecture
  - ☆ All requests are submitted to the presumed leader/primary
  - ☆ Thus, it's likely always the same primary that initiates Paxos for each position in the log
  - ☆ Thus, phase 1 can be omitted, if the primary is stable
  - ☆ Down to 3 message hops and logs
  - ☆ When primary fails, another one tries to take over
    - Has to perform the first phase to propose to become the leader
    - From then, it is the new stable primary leader….

What about reading a data item?

- ??